

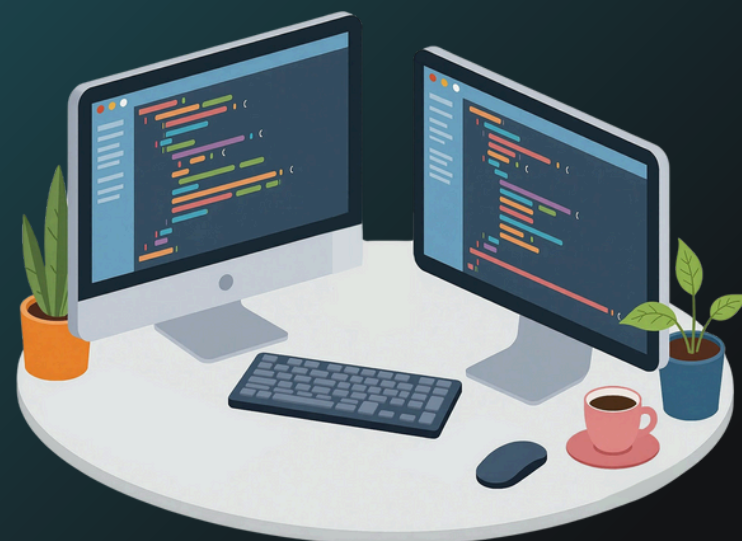
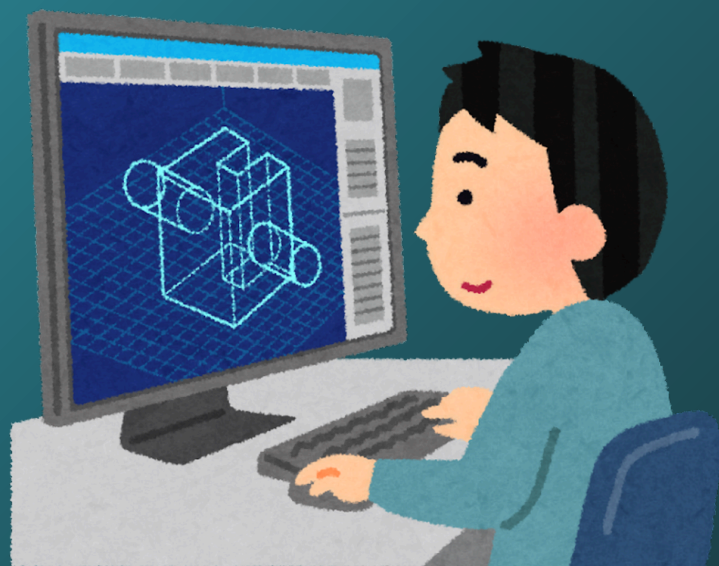
Natural Process Language

Development of a Naive Bayesian classifier for spam detection

Done by: Ayufkaliyev Y.
Checked by: Zhiyenbekov Zh.

introduction

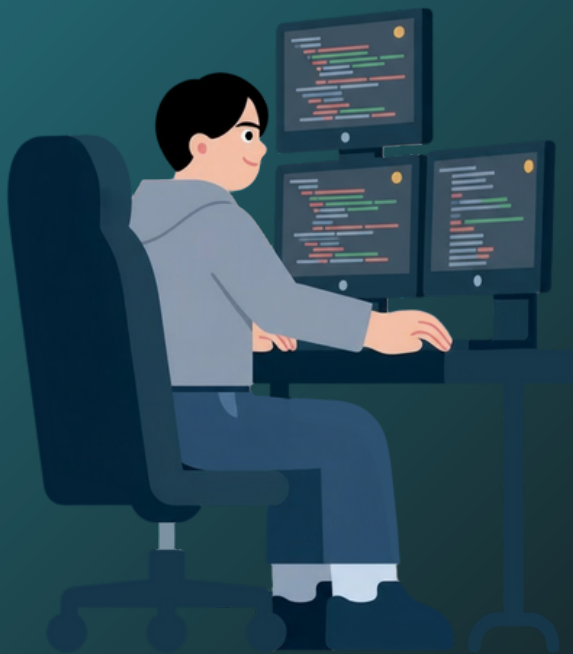
- Dataset: Data preparation cleaning and partitioning
- Model: Learning and prediction
- Main: Downloading data and launching the pipeline



Goals



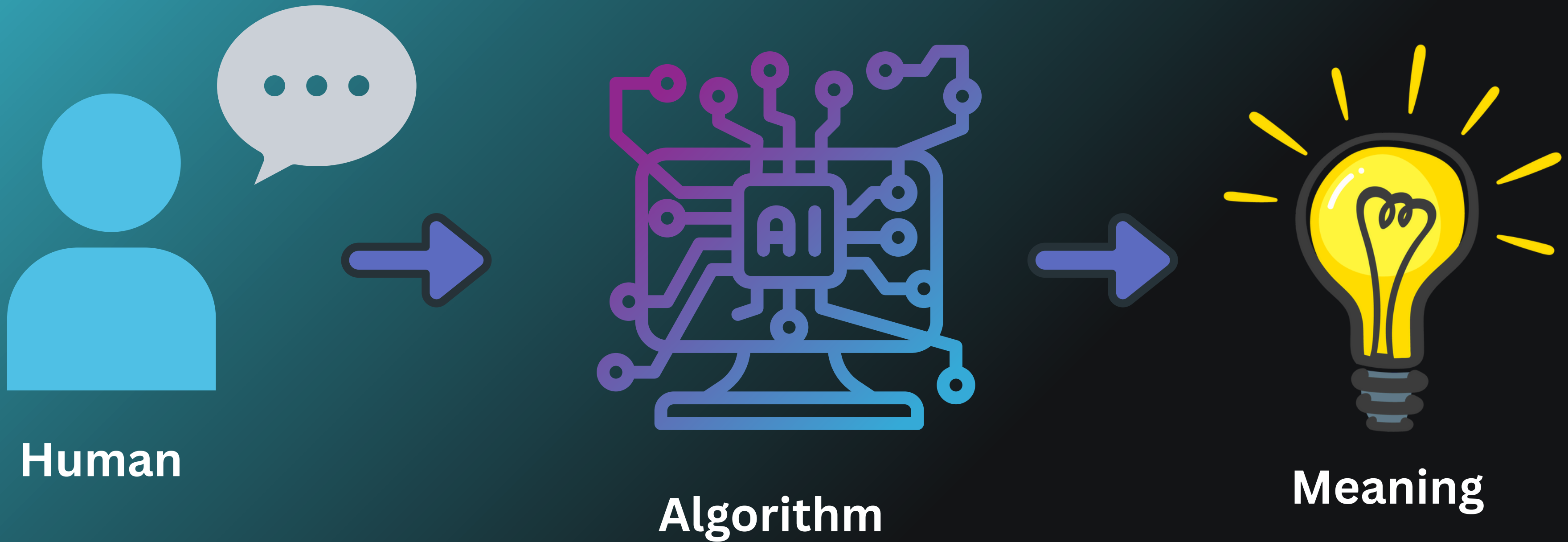
Create a machine learning model for classifying SMS messages into two classes "spam" and "ham" (not spam)



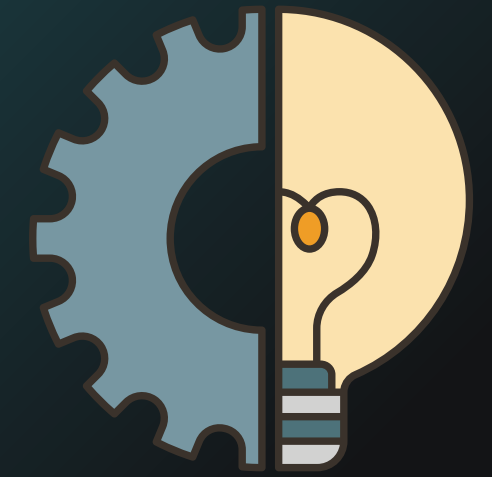
"NLP is a bridge between text and intelligence"

— Noam Chomsky

Project main idea



Model Development



We calculate the probability that the message is **spam based on the probability** of each word being found in spam.

Key words:

vocab: dictionary of all unique words

spam/ham: dictionaries of particularity

alpha: smoothing coefficient

Preprocessing

```
def fit(self, dataset):  
    ...  
    dataset - объект класса Dataset  
    Функция использует входной аргумент "dataset",  
    чтобы заполнить все атрибуты данного класса.  
    ...  
    # Начало вашего кода  
    self._train_X, self._train_y = dataset.train  
    self._val_X, self._val_y = dataset.val  
    self._test_X, self._test_y = dataset.test  
  
    self.label2num = dataset.label2num  
    self.num2label = dataset.num2label  
  
    for i, message in enumerate(self._train_X):  
        words = message.split()  
        label = self._train_y[i]  
  
        for word in words:  
            self.vocab.add(word)  
  
            if label == self.label2num["spam"]:  
                self.spam[word] = self.spam.get(word, 0) + 1  
            else:  
                self.ham[word] = self.ham.get(word, 0) + 1  
  
    self.Nvoc = len(self.vocab)  
    self.Nspam = sum(self.spam.values())
```



```
def inference(self, message):  
    ...  
    Функция принимает одно сообщение и, используя наивный байесовский алгоритм, определяет  
    ...  
    # Начало вашего кода  
    import re  
    cleaned = re.sub(r'^a-zA-Z\s', '', str(message).lower())  
    cleaned = ' '.join(cleaned.split())  
    words = cleaned.split()  
  
    total_messages = len(self._train_y)  
    p_spam_prior = np.sum(self._train_y == self.label2num["spam"]) / total_messages  
    p_ham_prior = np.sum(self._train_y == self.label2num["ham"]) / total_messages  
  
    log_pspam = np.log(p_spam_prior)  
    log_pham = np.log(p_ham_prior)  
  
    for word in words:  
        spam_count = self.spam.get(word, 0)  
        p_word_spam = (spam_count + self.alpha) / (self.Nspam + self.alpha * self.Nvoc)  
  
        ham_count = self.ham.get(word, 0)  
        p_word_ham = (ham_count + self.alpha) / (self.Nham + self.alpha * self.Nvoc)  
  
        log_pspam += np.log(p_word_spam)  
        log_pham += np.log(p_word_ham)
```

Key part of NLP implementation

Processing



The presentation is structured logically, includes visual explanations, and clearly demonstrates each stage of the NLP process.. Visual explanations and diagrams are used to illustrate complex NLP concepts in a simple and understandable way. Each stage of the NLP process, including data preprocessing, model implementation, and evaluation, is clearly demonstrated. The slides are well-organized and avoid unnecessary details, making the content easy to follow. Code examples and visual elements support the explanation of technical components. Overall, the presentation effectively communicates both the theoretical and practical aspects of the project.

Thank You

Zhalgas Zhiyenbekov the best professor